

DOCUMENTAÇÃO DA APLICAÇÃO

Automate-it Mini

Objetivos, arquitetura, páginas, funções, API, banco de dados e operação

Documento técnico-funcional

Este material descreve a versão atual do projeto encontrada no workspace. Ele pode ser usado como guia de apresentação, estudo, manutenção e testes manuais.

Preparado em 02 de outubro de 2026

1. Visão geral e índice

Objetivo da aplicação

O Automate-it Mini é uma aplicação web para cadastrar casos de teste de APIs, registrar ciclos de execução e acompanhar resultados. A versão atual gerencia o processo e seus estados; ela não dispara automaticamente requisições contra os endpoints cadastrados.

2	Arquitetura e fluxo de dados
3	Tecnologias utilizadas
4	Estrutura de pastas
5	Páginas e funções do frontend
6	Serviços de comunicação do frontend
7	Backend: controllers e serviços
8	Dados, DTOs e banco de dados
9	API REST
10	Ciclo de vida de uma execução
11	Botões e comportamentos
12	Validação, erros e segurança
13	Testes e operação
14	Limitações e próximos passos

Principais objetivos

- Centralizar a definição de casos de teste com nome, método HTTP, endpoint e descrição.
- Registrar execuções por projeto, ambiente, tipo, autor e observações.
- Controlar o ciclo de vida das execuções e impedir transições inválidas.
- Exibir indicadores e resultados usando dados persistidos no PostgreSQL.
- Disponibilizar uma API documentada pelo Swagger/OpenAPI para integração e teste.

2. Arquitetura e fluxo de dados

NAVEGADOR	API REST	PERSISTÊNCIA
React 19 App.jsx + CSS Fetch Services	Spring Boot 3.5 Controllers Services DTOs	Spring Data JPA Hibernate PostgreSQL 16
localhost:5173	localhost:8081/api	localhost:5432

A aplicação segue uma arquitetura cliente-servidor em camadas. O navegador renderiza a interface React. Os serviços JavaScript usam fetch para enviar JSON à API Spring Boot. Controllers recebem as requisições, services aplicam regras de negócio e repositories usam JPA/Hibernate para ler e gravar no PostgreSQL.

Fluxo principal

Etapas	Componente	Responsabilidade
1	Página React	Captura a ação do usuário e mantém o estado visual.
2	Service JS	Monta a requisição HTTP e interpreta sucesso ou erro.
3	Controller	Expõe a rota, desserializa e valida o corpo recebido.
4	Service Java	Normaliza dados, aplica regras e controla transações.
5	Repository/JPA	Executa persistência e consultas paginadas.
6	PostgreSQL	Mantém test_cases e test_runs de forma durável.
7	Resposta	DTO volta como JSON e atualiza a tela sem recarregar a página.

Decisão importante sobre Results

Results não possui entidade ou tabela própria. A tela filtra Test Runs cujo status é PASSED, FAILED ou CANCELLED. Assim, uma única execução é a fonte de verdade para estado, datas e resultado final.

3. Tecnologias utilizadas

Camada	Tecnologia	Função no projeto
Frontend	React 19.2.8	Componentes, estado, efeitos, formulários e renderização das páginas.
Build frontend	Vite 8.2	Servidor de desenvolvimento e geração do bundle de produção.
Qualidade frontend	ESLint 9	Análise estática do JavaScript e das regras de hooks.
Backend	Java 17 + Spring Boot 3.5	API REST, injeção de dependências, configuração e execução.
Web	Spring Web MVC	Controllers, JSON, status HTTP e CORS.
Persistência	Spring Data JPA + Hibernate	Repositories, mapeamento objeto-relacional e transações.
Validação	Jakarta Validation	Regras @NotBlank, @NotNull, @Size e @Pattern nos DTOs.
Banco	PostgreSQL 16	Armazenamento principal da aplicação.
Testes	Spring Boot Test + H2	Testes de integração isolados do banco de desenvolvimento.
Documentação API	Springdoc OpenAPI 2.8.9	Swagger UI e contrato navegável dos endpoints.
Infra local	Docker Compose	Criação do PostgreSQL com porta e volume persistente.

Endereços padrão

Recurso	Endereço
Frontend	http://localhost:5173
API	http://localhost:8081/api
Swagger UI	http://localhost:8081/swagger-ui/index.html
PostgreSQL	localhost:5432 / banco automateit

4. Estrutura de pastas

Pasta/arquivo	Objetivo
frontend/src/App.jsx	Componente principal, páginas, formulários, indicadores e ações da interface.
frontend/src/services	Funções que concentram a comunicação HTTP com a API.
frontend/src/assets	Logotipo e imagens estáticas usadas pela interface.
frontend/src/App.css	Estilos das páginas, cards, formulários, estados e responsividade.
frontend/public	Arquivos servidos diretamente pelo Vite.
backend/.../controller	Rotas REST de Test Cases e Test Runs.
backend/.../service	Regras de negócio, normalização, transações e conversão para DTO.
backend/.../repository	Interfaces Spring Data para acesso ao banco.
backend/.../entity	Mapeamento das tabelas test_cases e test_runs.
backend/.../dto	Contratos de entrada, saída e erro da API.
backend/.../enums	Valores válidos de ambiente, tipo de execução e status.
backend/.../exception	Exceções de domínio e resposta global de erros.
backend/.../config	Configuração CORS da API.
backend/src/main/resources	Configuração de conexão, Hibernate, porta e origens permitidas.
backend/src/test	Testes de contexto e integração dos controllers.
docker-compose.yml	PostgreSQL 16, credenciais locais, porta e volume postgres_data.

5. Páginas e funções do frontend

App

É o orquestrador da interface. Mantém página ativa, formulários, coleções de casos e execuções, indicadores de carregamento, salvamento e erro. Carrega dados ao iniciar e distribui callbacks aos componentes filhos.

Função/componente	Objetivo e comportamento
formatDate(value)	Converte data ISO para data/hora local; retorna um traço quando não existe valor.
loadData()	Busca casos e execuções em paralelo, limpa erros anteriores e controla o estado loading.
useEffect inicial	Faz a primeira carga e evita atualizar estado depois que o componente é desmontado.
results / useMemo	Filtra execuções finais para alimentar a página Results.
openTestCaseForm()	Navega para Test Cases, abre o formulário e limpa o erro atual.
openTestRunForm()	Navega para Test Runs, abre o formulário e limpa o erro atual.
handleCreateTestCase(event)	Impede reload, envia POST, inclui o registro retornado no topo e limpa/fecha o formulário.
handleCreateTestRun(event)	Envia uma nova execução, adiciona a resposta à lista e restaura o formulário.
handleStatusChange(run, status)	Envia PATCH e substitui somente a execução atualizada no estado local.
Dashboard	Mostra totais reais, aprovados, reprovados e as cinco execuções mais recentes.
TestCasesPage	Criação e listagem de casos de teste, método HTTP, endpoint e descrição.
TestRunsPage	Criação, listagem e controle do ciclo de vida das execuções.
ResultsPage	Apresenta somente execuções finalizadas, com metadados, datas e observações.
RunList	Lista execuções e escolhe botões permitidos conforme o status.
StatusBadge	Aplica classe visual específica ao status recebido.
PageHeader	Cabeçalho reutilizável com título, contexto e ação principal opcional.
EmptyState	Mensagem e ação contextual quando uma coleção ainda está vazia.
FormHeader	Título do formulário e botão de fechamento.
FormField	Wrapper acessível para rótulo e controle de entrada.
FormActions	Botões Cancel e Submit; desabilita envio enquanto saving estiver ativo.

6. Serviços de comunicação do frontend

Configuração comum

A variável `VITE_API_URL` pode apontar para outra API. Sem ela, o frontend usa `http://localhost:8081/api`. Cada módulo mantém a URL específica de seu recurso.

Função	Requisição	Resultado
<code>parseResponse(response)</code>	Qualquer resposta	Retorna JSON em sucesso. Em erro, reúne <code>validationErrors</code> ou usa <code>message</code> para criar um <code>Error</code> amigável.
<code>getTestCases()</code>	<code>GET /test-cases?page=0&size=100&sort=createdAt,desc</code>	Retorna o array <code>content</code> ; usa <code>[]</code> se o conteúdo não estiver presente.
<code>createTestCase(testCase)</code>	<code>POST /test-cases</code>	Serializa o formulário em JSON e devolve o caso criado.
<code>getTestRuns()</code>	<code>GET /test-runs?page=0&size=100&sort=createdAt,desc</code>	Retorna até 100 execuções ordenadas da mais recente para a mais antiga.
<code>createTestRun(testRun)</code>	<code>POST /test-runs</code>	Cria a execução inicialmente como <code>PENDING</code> .
<code>updateTestRunStatus(id, status)</code>	<code>PATCH /test-runs/{id}/status</code>	Envia <code>{status}</code> e devolve a execução já atualizada pelo backend.

Estado da interface

- `loading` controla a primeira carga e o botão `Refresh data`.
- `saving` evita envios e alterações simultâneas enquanto uma operação está em andamento.
- `error` alimenta um banner descartável no topo da área principal.
- `testCases` e `testRuns` são atualizados com a resposta real da API, sem recarregar o navegador.

7. Backend: controllers e serviços

Controllers

Método	Responsabilidade
TestCaseController.create	Valida CreateTestCaseRequest, chama o service e responde 201 Created.
TestCaseController.findAll	Lista casos paginados; padrão de 20 itens por createdAt decrescente.
TestCaseController.findById	Busca um caso pelo UUID.
TestRunController.create	Valida e cria uma execução; responde 201 Created.
TestRunController.findAll	Lista execuções paginadas e ordenadas por criação.
TestRunController.findById	Busca uma execução pelo UUID.
TestRunController.updateStatus	Valida o novo status e delega a transição ao service.

TestCaseService

Método	Responsabilidade
create	Remove espaços, padroniza método em maiúsculas, normaliza descrição e persiste.
findAll	Consulta o repository com paginação e converte cada entidade em resposta.
findById	Busca por UUID ou lança ResourceNotFoundException.
normalizeOptionalText	Transforma texto vazio em null ou retorna o texto sem espaços externos.
toResponse	Copia somente os campos públicos da entidade para TestCaseResponse.

TestRunService

Método	Responsabilidade
create	Normaliza campos, define PENDING e persiste a nova execução.
findAll / findById	Lista ou recupera execução; inexistência gera 404.
updateStatus	Valida transição, registra startedAt/finishedAt, marca finished e salva.
validateTransition	Aceita status idêntico ou consulta ALLOWED_TRANSITIONS; caso contrário gera conflito 409.
isFinalStatus	Reconhece PASSED, FAILED e CANCELLED como estados finais.
findEntityById	Centraliza a busca da entidade e a exceção de não encontrado.
normalizeOptionalText / toResponse	Normaliza notas e cria o DTO devolvido ao cliente.

8. Dados, DTOs e banco de dados

Tabela test_cases

Campo	Tipo lógico	Regra/uso
id	UUID	Chave primária gerada automaticamente.
name	texto(150)	Nome obrigatório.
method	texto(10)	GET, POST, PUT, PATCH ou DELETE.
endpoint	texto(500)	Obrigatório e iniciado por /.
description	texto(2000)	Descrição opcional.
created_at / updated_at	data e hora	Preenchidas por callbacks JPA.

Tabela test_runs

Campo	Tipo lógico	Regra/uso
id	UUID	Chave primária gerada automaticamente.
name	texto(150)	Nome da execução.
project_name	texto(150)	Projeto associado.
environment	enum	DEV, QA, STAGING ou PROD_SIMULATED.
execution_type	enum	SMOKE, REGRESSION, SANITY, API ou UI.
status	enum	PENDING, RUNNING, PASSED, FAILED ou CANCELLED.
created_by	texto(150)	Responsável informado na criação.
notes	texto(2000)	Observações opcionais.
finished	booleano	Verdadeiro ao chegar a um status final.
created_at / updated_at	data e hora	Auditoria básica de criação e alteração.
started_at / finished_at	data e hora	Marcadas automaticamente nas transições.

DTOs

CreateTestCaseRequest, CreateTestRunRequest e UpdateTestRunStatusRequest validam as entradas. TestCaseResponse e TestRunResponse definem o JSON de saída. ApiError padroniza timestamp, status, nome do erro, mensagem, rota e erros por campo. Getters e setters das entidades/DTOs sustentam a serialização e o mapeamento, sem regra de negócio própria.

9. API REST

Método e rota	Uso	Resposta esperada
POST /api/test-cases	Cadastrar caso de teste.	201 + TestCaseResponse
GET /api/test-cases	Listar com page, size e sort.	200 + Page
GET /api/test-cases/{id}	Buscar caso por UUID.	200 ou 404
POST /api/test-runs	Criar execução PENDING.	201 + TestRunResponse
GET /api/test-runs	Listar execuções paginadas.	200 + Page
GET /api/test-runs/{id}	Buscar execução por UUID.	200 ou 404
PATCH /api/test-runs/{id}/status	Alterar status dentro das regras.	200, 404 ou 409

Exemplos de JSON

```
POST /api/test-cases
{
  "name": "Login API",
  "method": "POST",
  "endpoint": "/api/login",
  "description": "Valida autenticação"
}
```

```
POST /api/test-runs
{
  "name": "Smoke QA",
  "projectName": "Automate-it Mini",
  "environment": "QA",
  "executionType": "SMOKE",
  "createdBy": "Analista",
  "notes": "Execução manual"
}
```

```
PATCH /api/test-runs/{id}/status
{ "status": "RUNNING" }
```

10. Ciclo de vida de uma execução

Toda execução nasce como PENDING. O backend, e não apenas a interface, protege as transições permitidas.

Estado atual	Próximos estados	Efeito
PENDING	RUNNING ou CANCELLED	RUNNING preenche startedAt; CANCELLED finaliza.
RUNNING	PASSED, FAILED ou CANCELLED	Preenche finishedAt e marca finished=true.
PASSED	Nenhum	Resultado final aprovado.
FAILED	Nenhum	Resultado final reprovado.
CANCELLED	Nenhum	Resultado final cancelado.

Mapa visual

PENDING	Start	RUNNING	Pass	PASSED
PENDING	Cancel	CANCELLED		
RUNNING	Fail	FAILED		
RUNNING	Cancel	CANCELLED		

Se o cliente tentar, por exemplo, alterar PASSED para RUNNING, InvalidStatusTransitionException é convertida em HTTP 409 Conflict.

11. Botões e comportamentos

Botão	Onde aparece	Função
Dashboard / Test Cases / Test Runs / Results	Barra lateral	Troca a página ativa sem recarregar o navegador.
+ New Test	Dashboard	Abre a página e o formulário de Test Case.
+ New Test Case	Test Cases	Exibe o formulário de cadastro.
+ New Test Run	Test Runs	Exibe o formulário de execução.
Save Test Case	Formulário	Valida no navegador, envia POST e atualiza a lista.
Create Test Run	Formulário	Envia POST e inclui a execução PENDING.
Cancel / X	Formulários	Fecha o formulário sem enviar os dados.
Start	Run PENDING	Altera o status para RUNNING.
Pass	Run RUNNING	Finaliza como PASSED.
Fail	Run RUNNING	Finaliza como FAILED.
Cancel	Run não finalizado	Finaliza como CANCELLED.
View result	Run finalizado	Navega para Results.
View all	Dashboard	Navega para a lista completa de execuções.
View all runs	Results	Retorna para Test Runs.
Create your first run	Estado vazio	Abre o formulário de nova execução.
Refresh data	Área principal	Refaz as duas consultas ao backend.
X do banner	Mensagem de erro	Remove o erro atual da tela.

12. Validação, erros e segurança

Validações

- Campos obrigatórios são verificados no frontend com required e novamente no backend.
- Comprimentos máximos são coerentes com DTOs e colunas do banco.
- Endpoint deve começar com /; método HTTP deve pertencer à lista permitida.
- Enums rejeitam ambiente, tipo ou status desconhecido.
- O backend remove espaços externos e converte campos opcionais vazios em null.

Tratamento global

Situação	HTTP	Tratamento
Recurso inexistente	404	ResourceNotFoundException com mensagem e rota.
Transição inválida	409	InvalidStatusTransitionException.
Falha de validação	400	Mapa validationErrors por campo.
JSON malformado/enum inválido	400	Mensagem de corpo inválido.
Parâmetro com tipo incorreto	400	Indica o nome do parâmetro.
Erro não previsto	500	Mensagem genérica sem expor detalhes internos.

Segurança atual

CORS permite apenas as origens configuradas para /api/** e os métodos GET, POST, PATCH e OPTIONS. A aplicação ainda não implementa autenticação, autorização, TLS, controle de perfil, auditoria completa ou proteção por usuário. Portanto, a configuração atual é adequada para desenvolvimento local, não para exposição pública sem endurecimento adicional.

13. Testes e operação

Cobertura existente

- BackendApplicationTests verifica a inicialização do contexto Spring.
- TestCaseControllerIntegrationTests cobre criação, consulta e validação de Test Cases.
- TestRunControllerIntegrationTests cobre criação, consulta, transições e erros de Test Runs.
- H2 é usado nos testes para isolar o PostgreSQL de desenvolvimento.
- ESLint e vite build verificam qualidade e empacotamento do frontend.

Inicialização local

Etapa	Comando / ação
1. Banco	Na raiz: docker compose up -d postgres
2. Backend	Em backend/backend: ./mvnw spring-boot:run
3. Frontend	Em frontend: npm install (primeira vez) e npm run dev
4. Interface	Abrir http://localhost:5173
5. Swagger	Abrir http://localhost:8081/swagger-ui/index.html

Variáveis de ambiente

Variável	Padrão	Uso
DB_URL	jdbc:postgresql://localhost:5432/automateit	Conexão JDBC.
DB_USERNAME	automateit	Usuário do banco.
DB_PASSWORD	automateit123	Senha local.
HIBERNATE_DDL_AUTO	update	Atualização do esquema.
SERVER_PORT	8081	Porta da API.
CORS_ALLOWED_ORIGINS	localhost/127.0.0.1:5173	Origens do frontend.
VITE_API_URL	http://localhost:8081/api	Base da API no frontend.

14. Limitações e próximos passos

O que a versão atual faz

- Cadastra definições de testes e registra execuções com histórico de status.
- Persiste dados, expõe API e apresenta indicadores e resultados finais.
- Permite conduzir manualmente uma execução do início ao resultado.

O que ainda não faz

- Não executa automaticamente chamadas HTTP contra o endpoint do Test Case.
- Test Runs ainda não se relacionam a Test Cases por chave estrangeira.
- Não armazena passos, assertions, corpo de resposta, evidências, duração calculada ou logs.
- Não oferece edição/exclusão, filtros avançados, busca ou paginação visual além dos 100 itens buscados.
- Não possui autenticação, usuários persistidos, permissões ou segregação por projeto.

Melhorias recomendadas

Prioridade	Melhoria	Benefício
Alta	Relacionar TestRun, TestCase e TestResult	Rastreabilidade entre definição, execução e evidência.
Alta	Motor de execução HTTP + assertions	Transforma o gerenciador em ferramenta real de automação.
Alta	Autenticação e autorização	Protege dados e ações por usuário/projeto.
Média	Flyway ou Liquibase	Versiona o esquema em vez de depender de ddl-auto=update.
Média	Filtros, busca e paginação no frontend	Escala para volumes maiores.
Média	Testes unitários/frontend e E2E	Reduz regressões na interface e nos fluxos completos.
Média	Observabilidade e logs estruturados	Facilita diagnóstico em ambientes implantados.
Baixa	Exportação PDF/CSV de resultados	Compartilhamento de relatórios com stakeholders.

Resumo final

O Automate-it Mini possui uma base coerente para gestão de testes: frontend reativo, API em camadas, validação, persistência e controle de estado. Seu próximo salto funcional é ligar cada execução a casos de teste e automatizar a chamada/avaliação dos endpoints, preservando o modelo atual como camada de gestão e histórico.